



神经网络和反向传播

NNs and Back Propagation



任课教师：孔雨秋

yqkong@dlut.edu.cn



- 01 多层感知机 (Multi-layer Perceptron)**
- 02 前馈神经网络 (Feedforward Neural Network)**
- 03 梯度反向传播 (Gradient Back Propagation)**

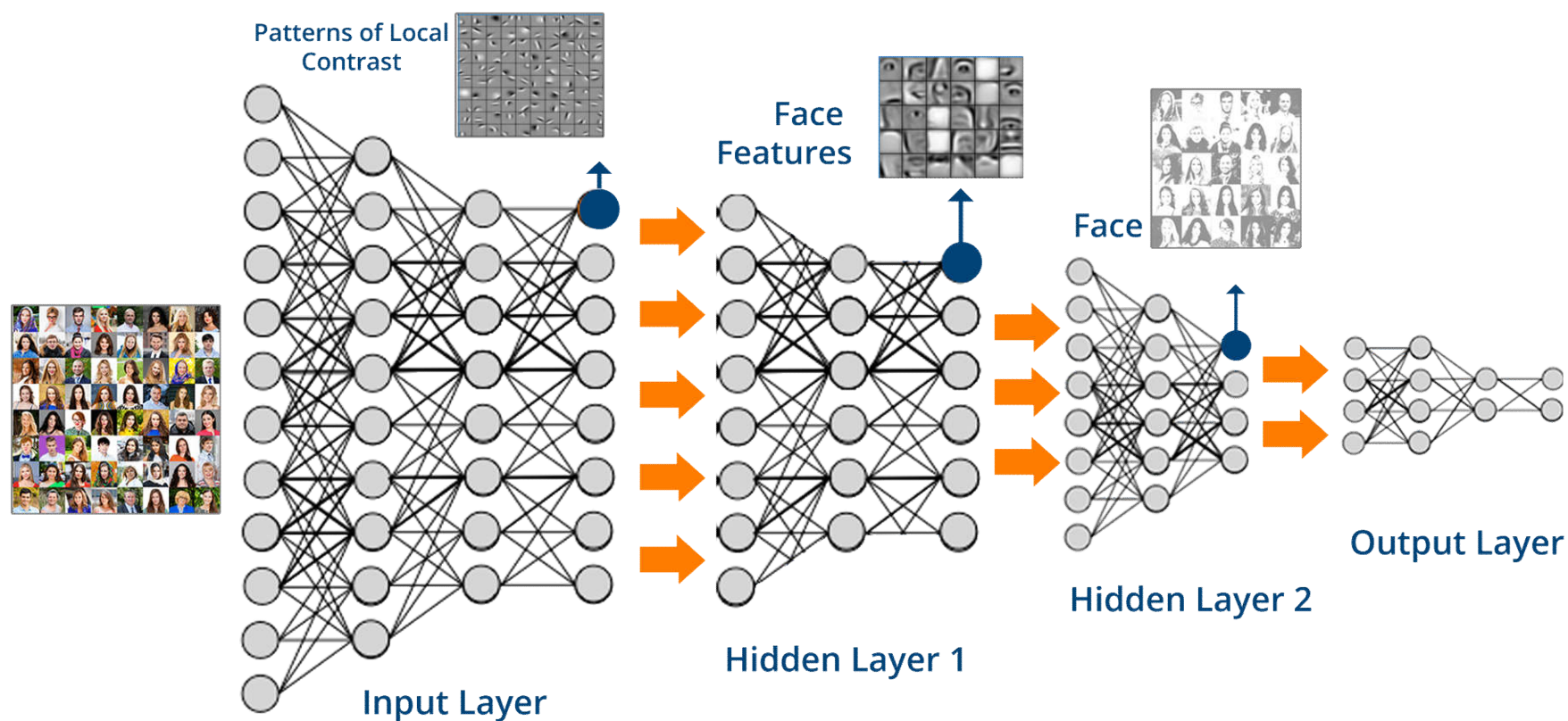


PART ONE

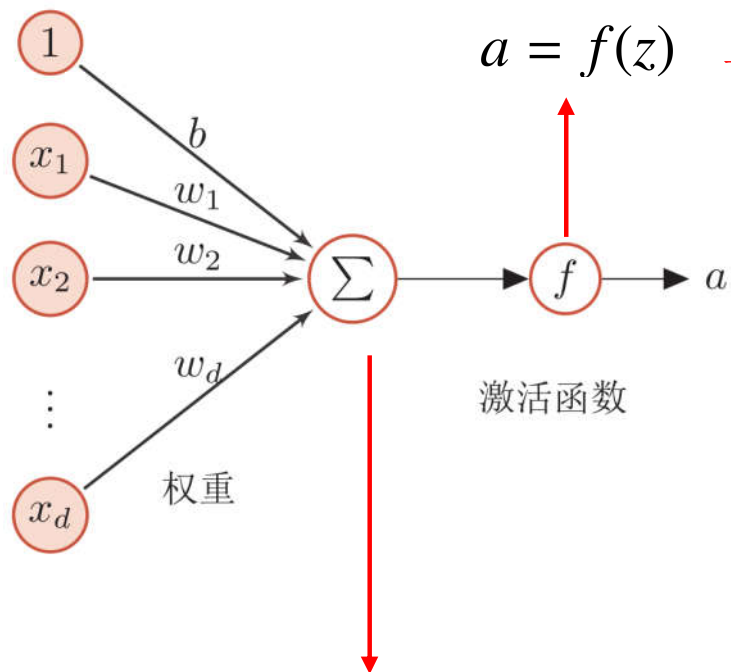
多层感知机

Multi-layer Perceptron

■ 深度神经网络

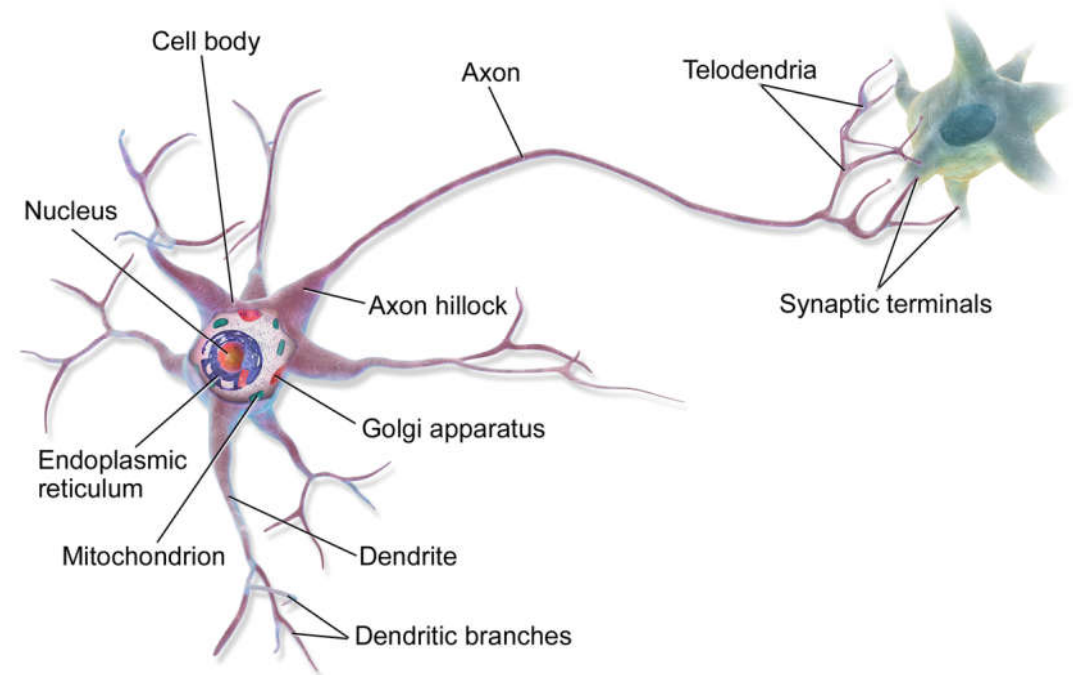


■ 单层感知机结构



$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

一个简单的线性模型！



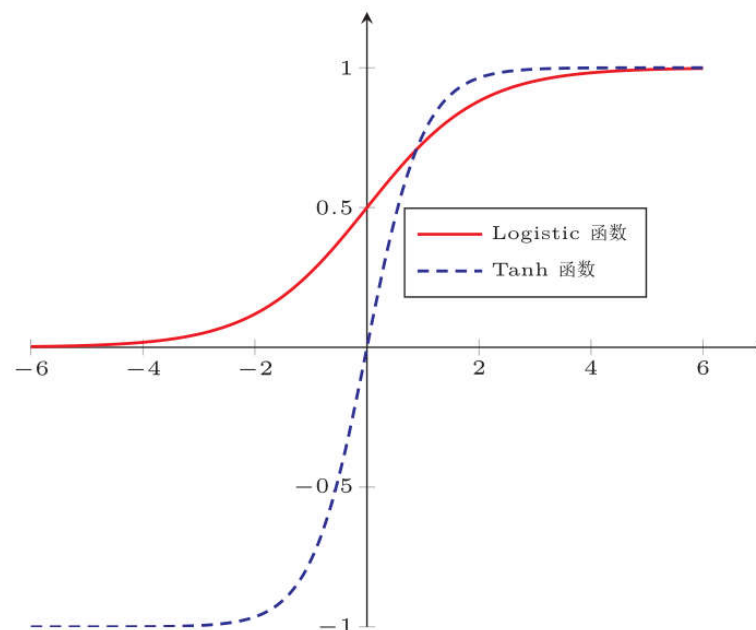
■ 常见激活函数

□ Sigmoid激活函数

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

□ Tanh激活函数

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



□ 性质

- ✓ 饱和函数：自变量绝对值较大的时候，值接近；
- ✓ tanh函数是零中心化的，而logistic函数的输出恒大于0；
- ✓ 非零中心化的输出会使得其后的神经元的输入发生偏置偏移（bias shift），并进一步使得梯度下降的收敛速度变慢。

■ 常见激活函数

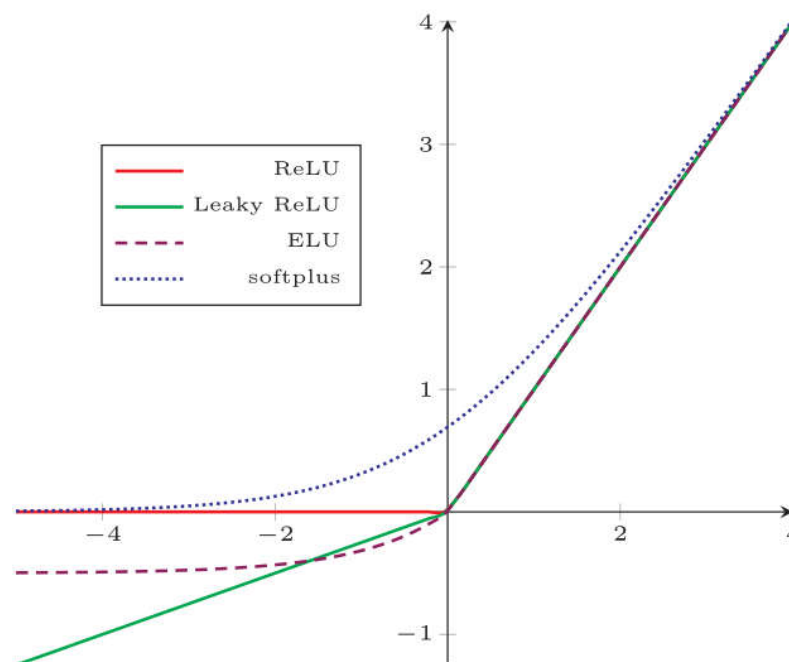
$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$

$$\text{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases}$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

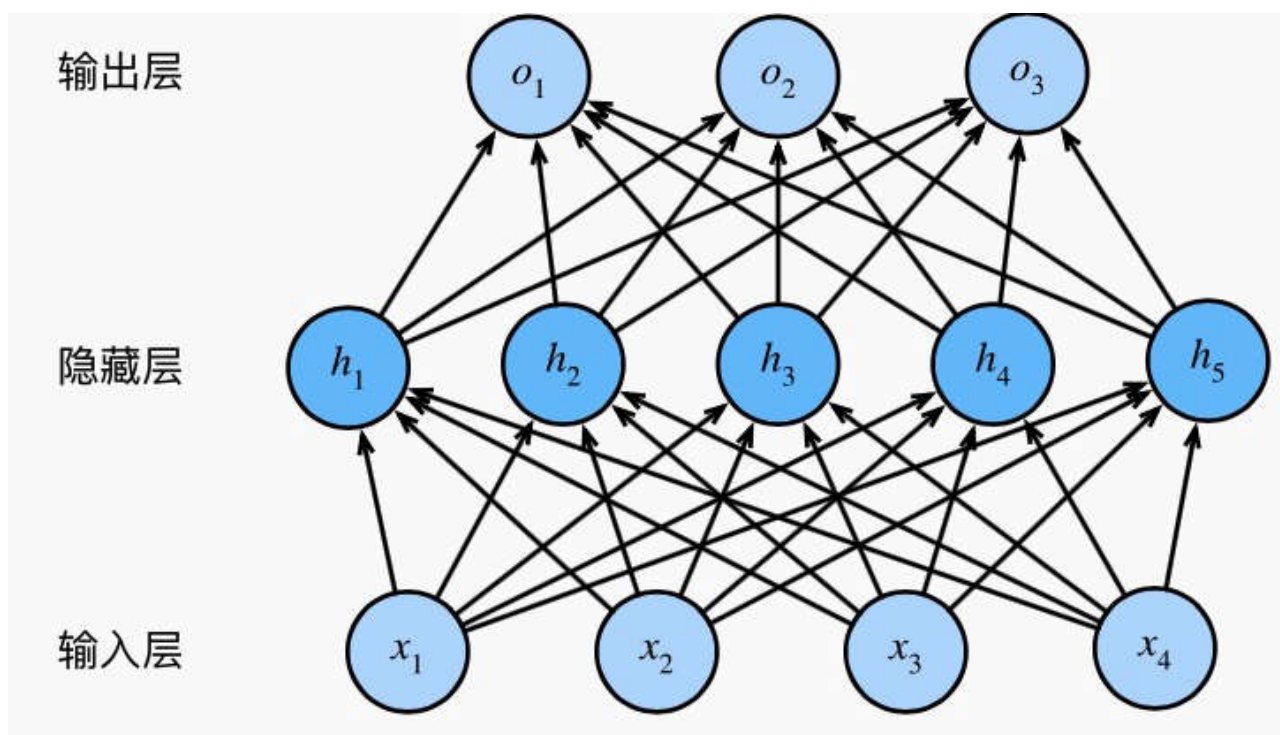
$$\text{softplus}(x) = \log(1 + \exp(x))$$



死亡ReLU问题 (Dying ReLU Problem)

- 计算上更加高效
- 生物学合理性
- 单侧抑制、宽兴奋边界
- 在一定程度上缓解梯度消失问题

- 多层感知机在单层感知机的基础上引入了一到多个隐藏层(hidden layer)。隐藏层位于输入层和输出层之间。不难发现，即便再添加更多的隐藏层，以上设计依然只能与仅含输出层的单层感知机等价。
- 线性变换下，等价于一个单层感知机



$$\mathbf{O} = (\mathbf{X}\mathbf{W}_h + \mathbf{b}_h)\mathbf{W}_o + \mathbf{b}_o = \mathbf{X}\mathbf{W}_h\mathbf{W}_o + \mathbf{b}_h\mathbf{W}_o + \mathbf{b}_o.$$

02

PART TWO

前馈神经网络 Feedforward NNs

■ 人工神经网络 (Artificial Neural Network)

人工神经网络主要由大量的神经元以及它们之间的有向连接构成。
因此考虑三方面：

□ 神经元的激活规则

主要是指神经元输入到输出之间的映射关系，一般为非线性函数。

□ 网络的拓扑结构

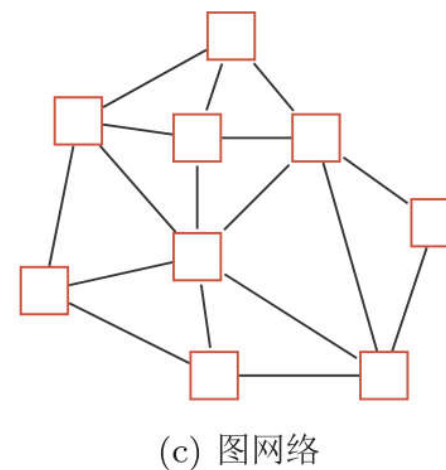
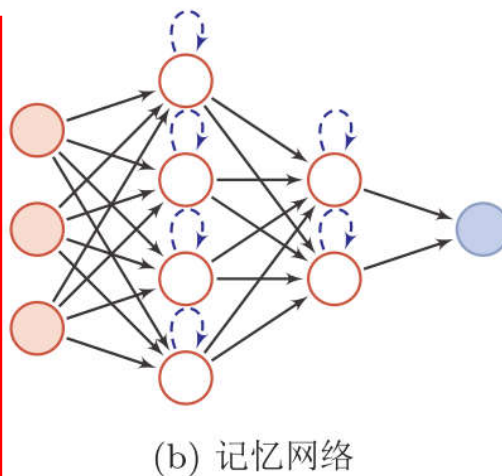
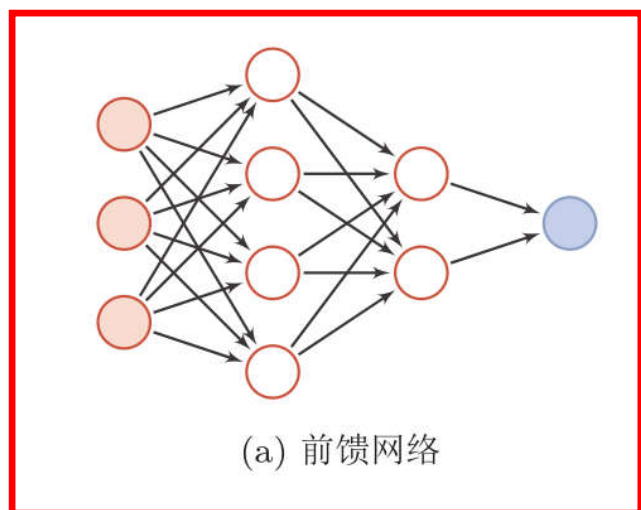
不同神经元之间的连接关系

□ 学习算法

通过训练数据来学习神经网络的参数

■ 前馈神经网络 (Feedforward Neural Networks)

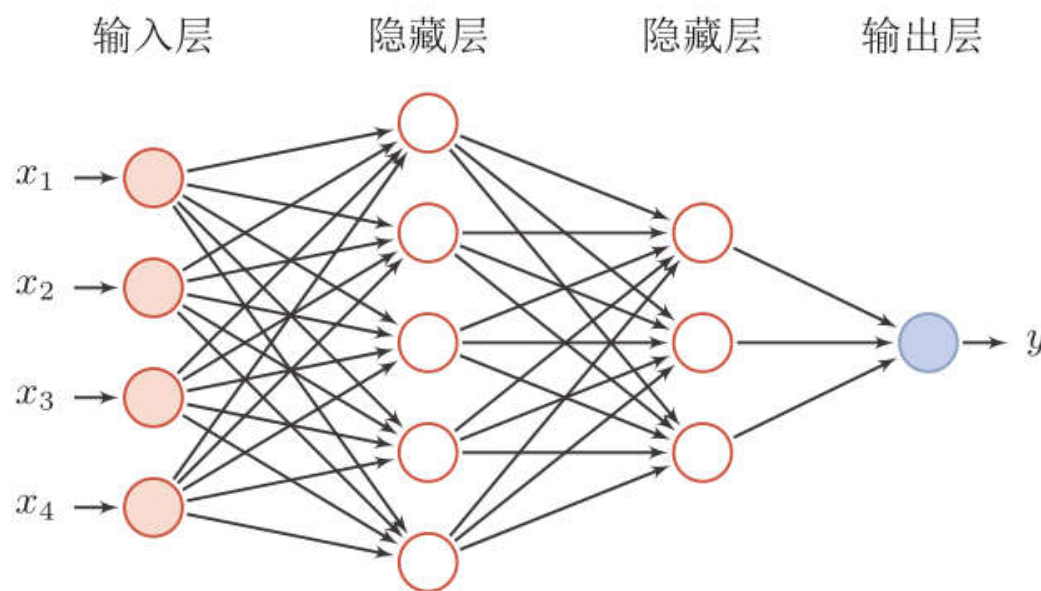
- 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。
- 前馈神经网络是人工神经网络的一种形式，信息前向传递，实现逐层编码处理的网络结构。



圆形节点表示一个神经元，方形节点表示一组神经元。

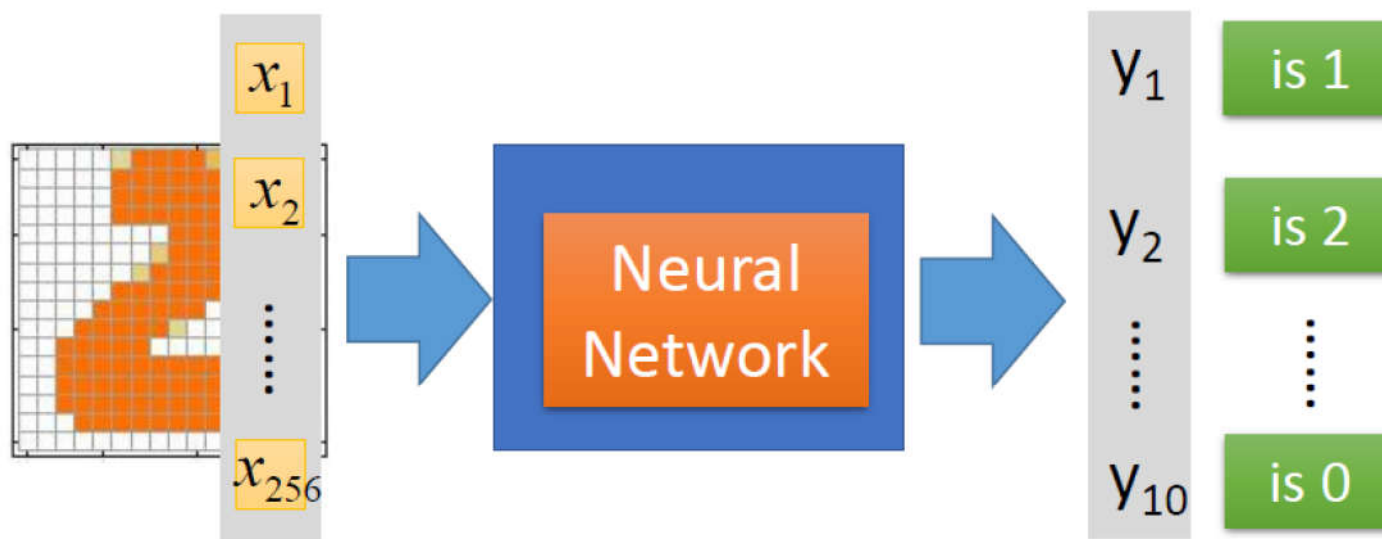
■ 前馈神经网络（全连接神经网络、多层感知器）

- 各神经元分别属于不同的层，层内无连接。
- 相邻两层之间的神经元全部两两连接。
- 整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。



记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

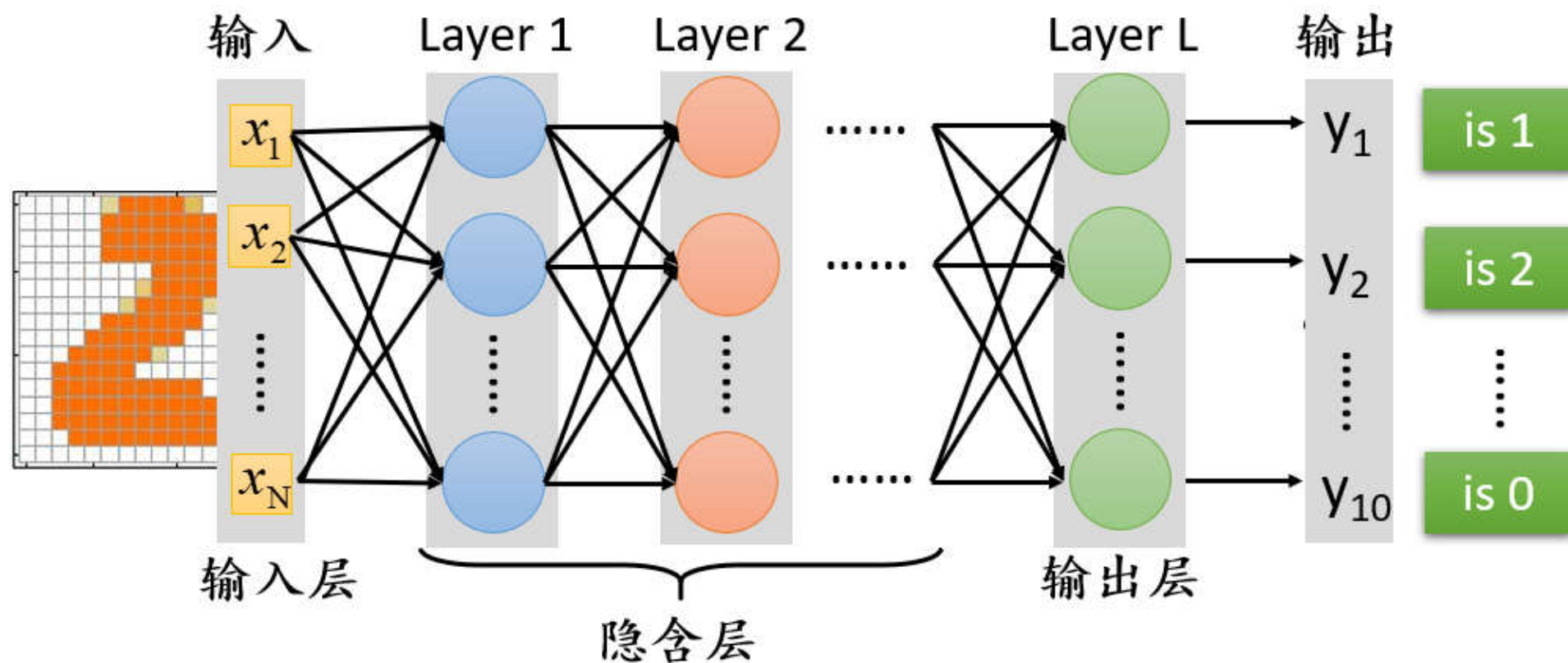
■ 例：手写数字识别



输入：256维向量

输出：10维向量

■ 例：手写数字识别

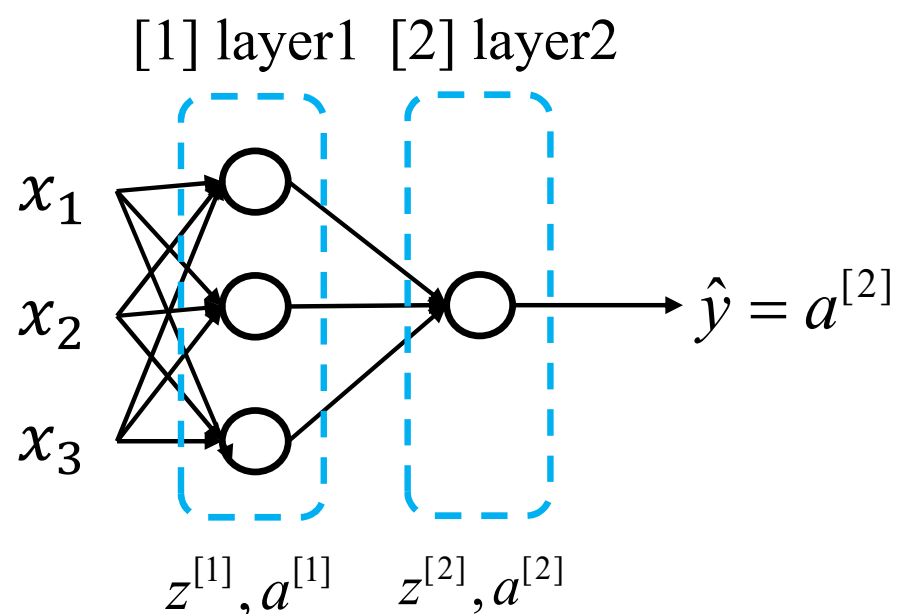


对网络结构进行合理设计，使其函数集合中能够有一个较好的函数

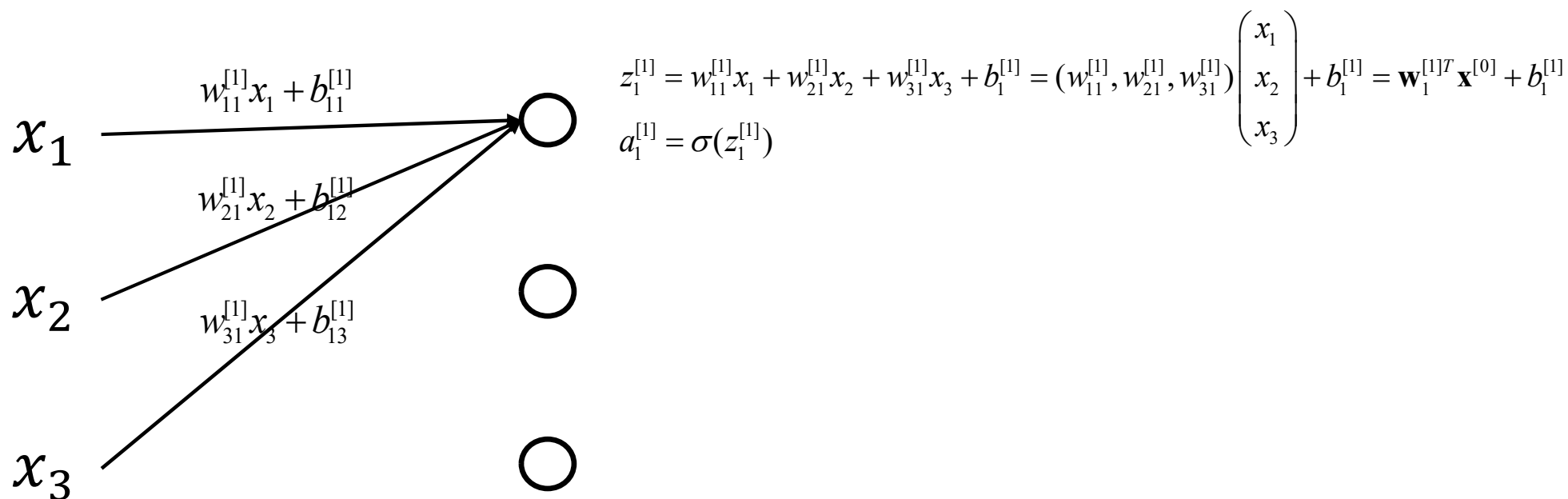
■ 单层感知机

$$\left. \begin{matrix} x \\ w \\ b \end{matrix} \right\} \rightarrow z = w^T x + b \rightarrow \hat{y} = \sigma(z) \rightarrow L(\hat{y}, y)$$

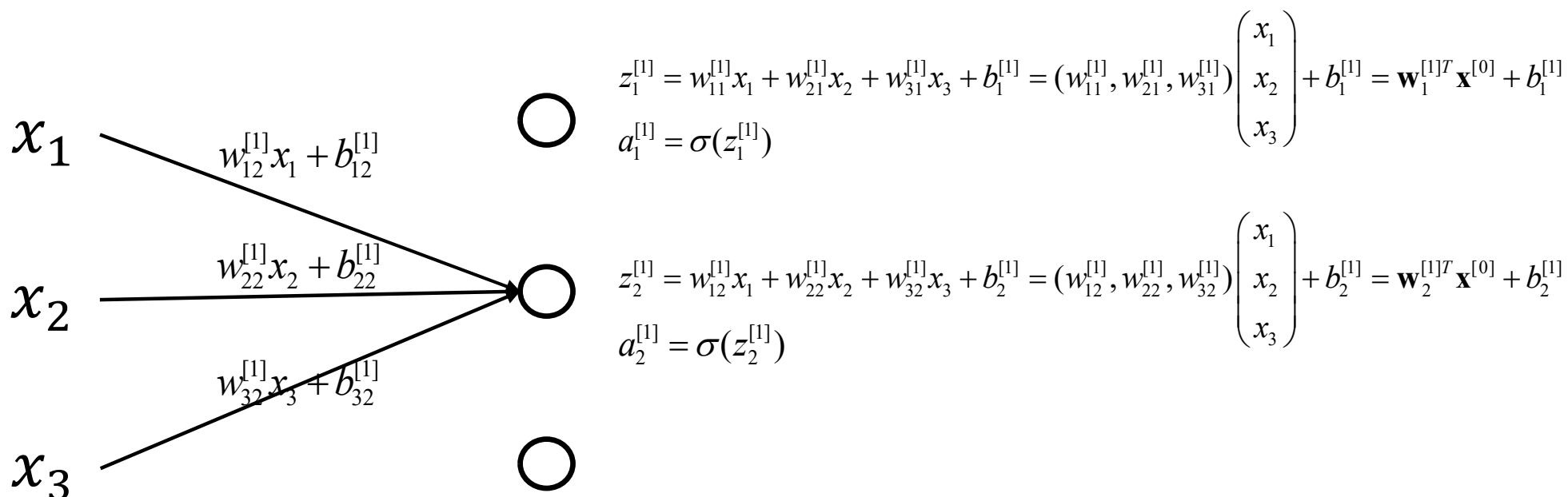
■ 神经网络模型



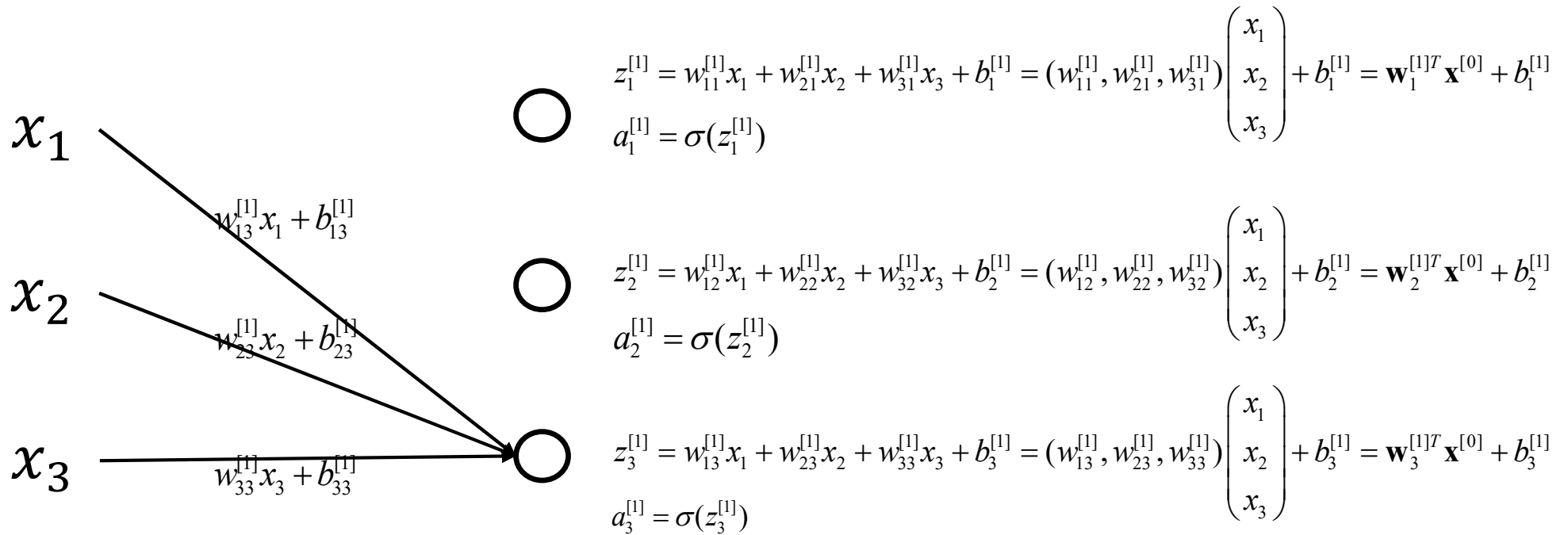
■ 第一层



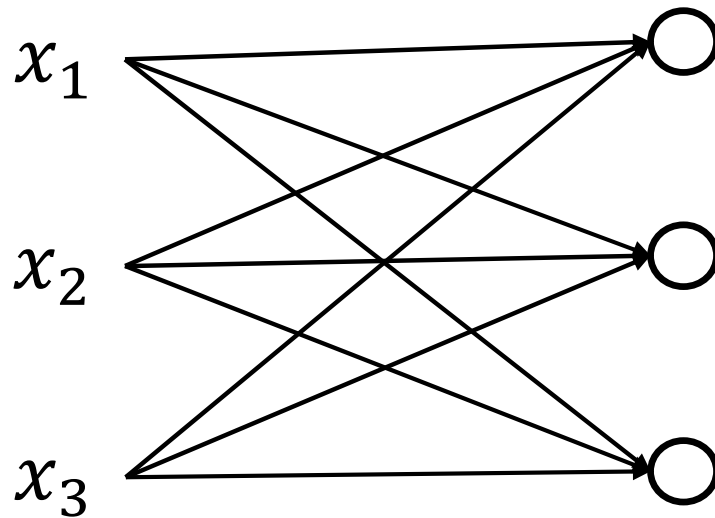
■ 第一层



■ 第一层



■ 第一层



$$z_1^{[1]} = w_{11}^{[1]}x_1 + w_{21}^{[1]}x_2 + w_{31}^{[1]}x_3 + b_1^{[1]} = (w_{11}^{[1]}, w_{21}^{[1]}, w_{31}^{[1]}) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + b_1^{[1]} = \mathbf{w}_1^{[1]T} \mathbf{x}^{[0]} + b_1^{[1]}$$
$$a_1^{[1]} = \sigma(z_1^{[1]})$$

$$z_2^{[1]} = w_{12}^{[1]}x_1 + w_{22}^{[1]}x_2 + w_{32}^{[1]}x_3 + b_2^{[1]} = (w_{12}^{[1]}, w_{22}^{[1]}, w_{32}^{[1]}) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + b_2^{[1]} = \mathbf{w}_2^{[1]T} \mathbf{x}^{[0]} + b_2^{[1]}$$
$$a_2^{[1]} = \sigma(z_2^{[1]})$$

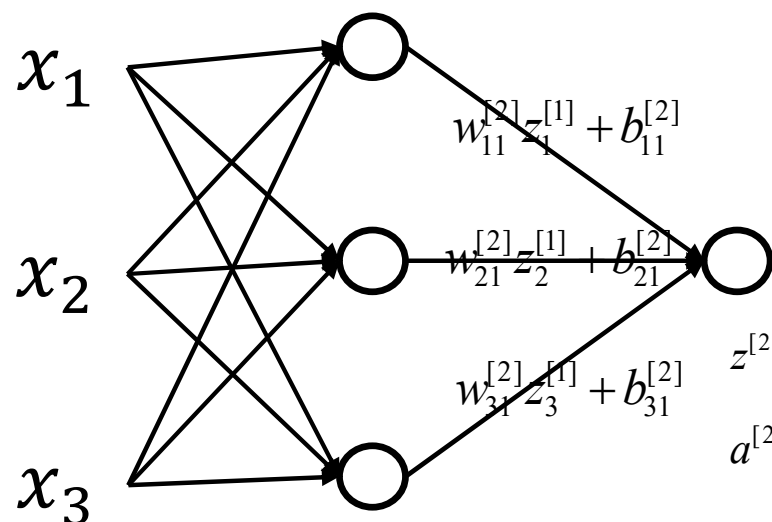
$$z_3^{[1]} = w_{13}^{[1]}x_1 + w_{23}^{[1]}x_2 + w_{33}^{[1]}x_3 + b_3^{[1]} = (w_{13}^{[1]}, w_{23}^{[1]}, w_{33}^{[1]}) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + b_3^{[1]} = \mathbf{w}_3^{[1]T} \mathbf{x}^{[0]} + b_3^{[1]}$$
$$a_3^{[1]} = \sigma(z_3^{[1]})$$

$$\begin{pmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \end{pmatrix} = \begin{pmatrix} w_{11}^{[1]} & w_{21}^{[1]} & w_{31}^{[1]} \\ w_{12}^{[1]} & w_{22}^{[1]} & w_{32}^{[1]} \\ w_{13}^{[1]} & w_{23}^{[1]} & w_{33}^{[1]} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \end{pmatrix}$$

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]T} \mathbf{z}^{[0]} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]})$$

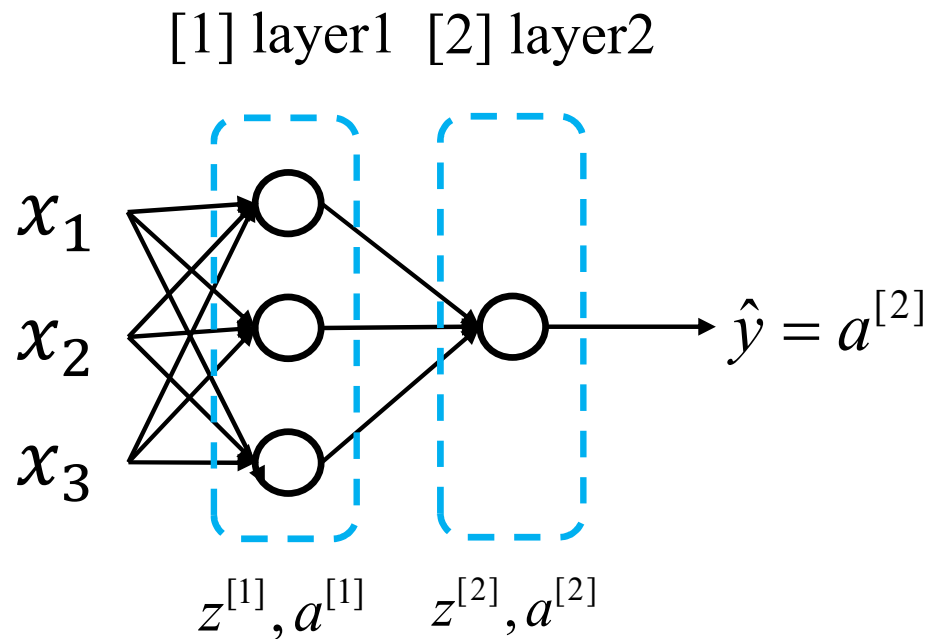
■ 第二层



$$z^{[2]} = w_{11}^{[2]} z_1^{[1]} + w_{21}^{[2]} z_2^{[1]} + w_{31}^{[2]} z_3^{[1]} + b_1^{[2]} = (w_{11}^{[2]}, w_{21}^{[2]}, w_{31}^{[2]}) \begin{pmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \end{pmatrix} + b_1^{[2]} = \mathbf{w}_1^{[2]} \mathbf{z}^{[2]} + b_1^{[2]}$$
$$a^{[2]} = \sigma(z^{[2]})$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]T} \mathbf{a}^{[1]} + \mathbf{b}^{[2]}$$

■ 第L层



$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]T} \mathbf{z}^{[0]} + \mathbf{b}^{[1]} \quad \text{第一层}$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]T} \mathbf{a}^{[1]} + \mathbf{b}^{[2]} \quad \text{第二层}$$

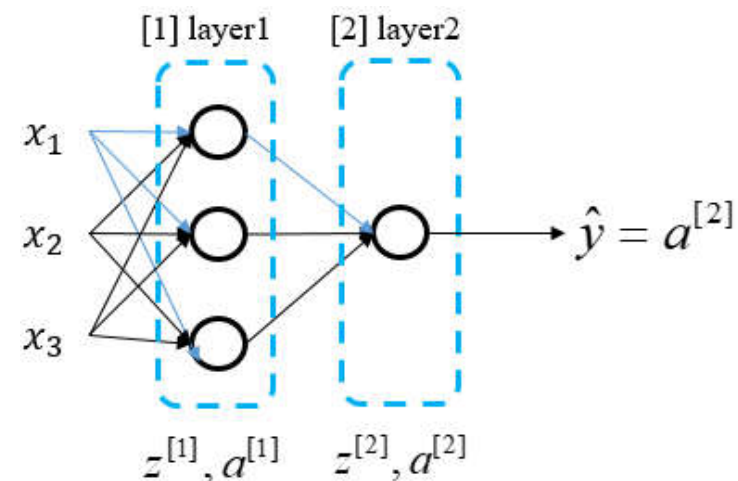
.....

.....

$$\mathbf{z}^{[L]} = \mathbf{W}^{[L]T} \mathbf{a}^{[L-1]} + \mathbf{b}^{[L]} \quad \text{第L层}$$

■ 第一层

$$\left. \begin{array}{l} \mathbf{x} \\ \mathbf{W}^{[1]} \\ \mathbf{b}^{[1]} \end{array} \right\} \rightarrow \mathbf{z}^{[1]} = \mathbf{W}^{[1]T} \mathbf{x} + \mathbf{b}^{[1]} \rightarrow \mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]})$$



■ 第二层

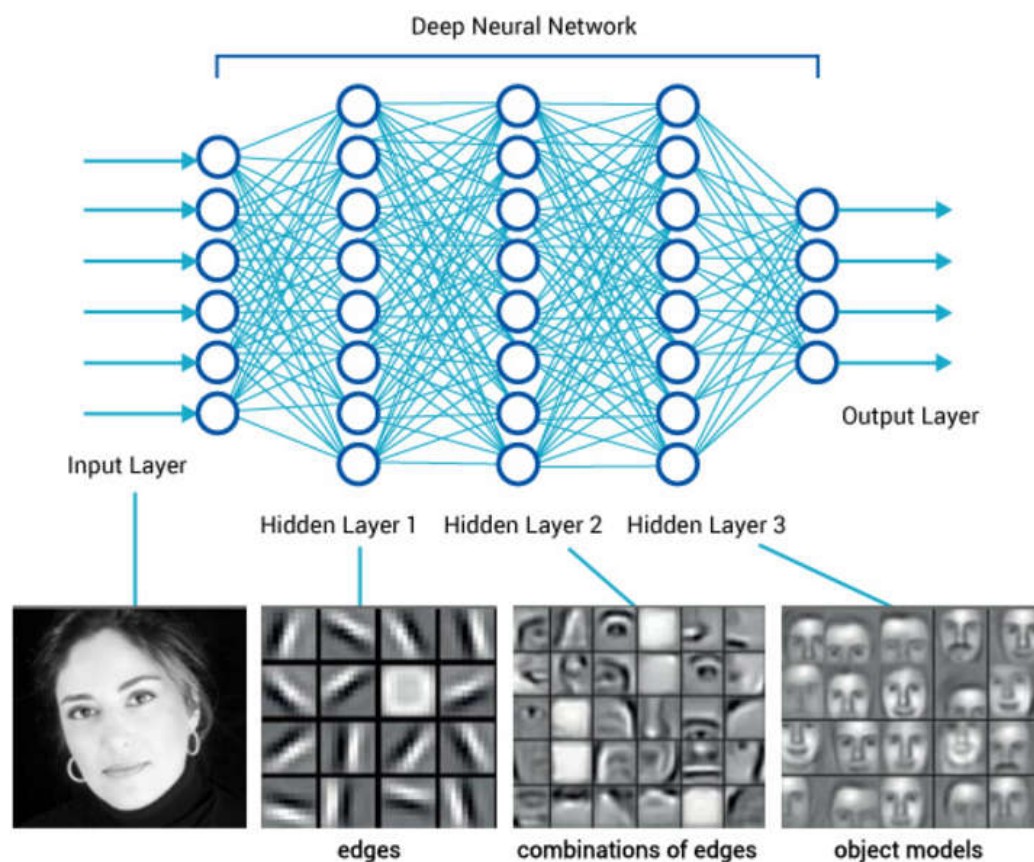
$$\left. \begin{array}{l} \mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]}) \\ \mathbf{W}^{[2]} \\ \mathbf{b}^{[2]} \end{array} \right\} \rightarrow \mathbf{z}^{[2]} = \mathbf{W}^{[2]T} \mathbf{a}^{[1]} + \mathbf{b}^{[2]} \rightarrow \mathbf{a}^{[2]} = \sigma(\mathbf{z}^{[2]}) \rightarrow L(\mathbf{a}^{[2]}, y)$$

■ 前馈计算—信息向前传递

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$



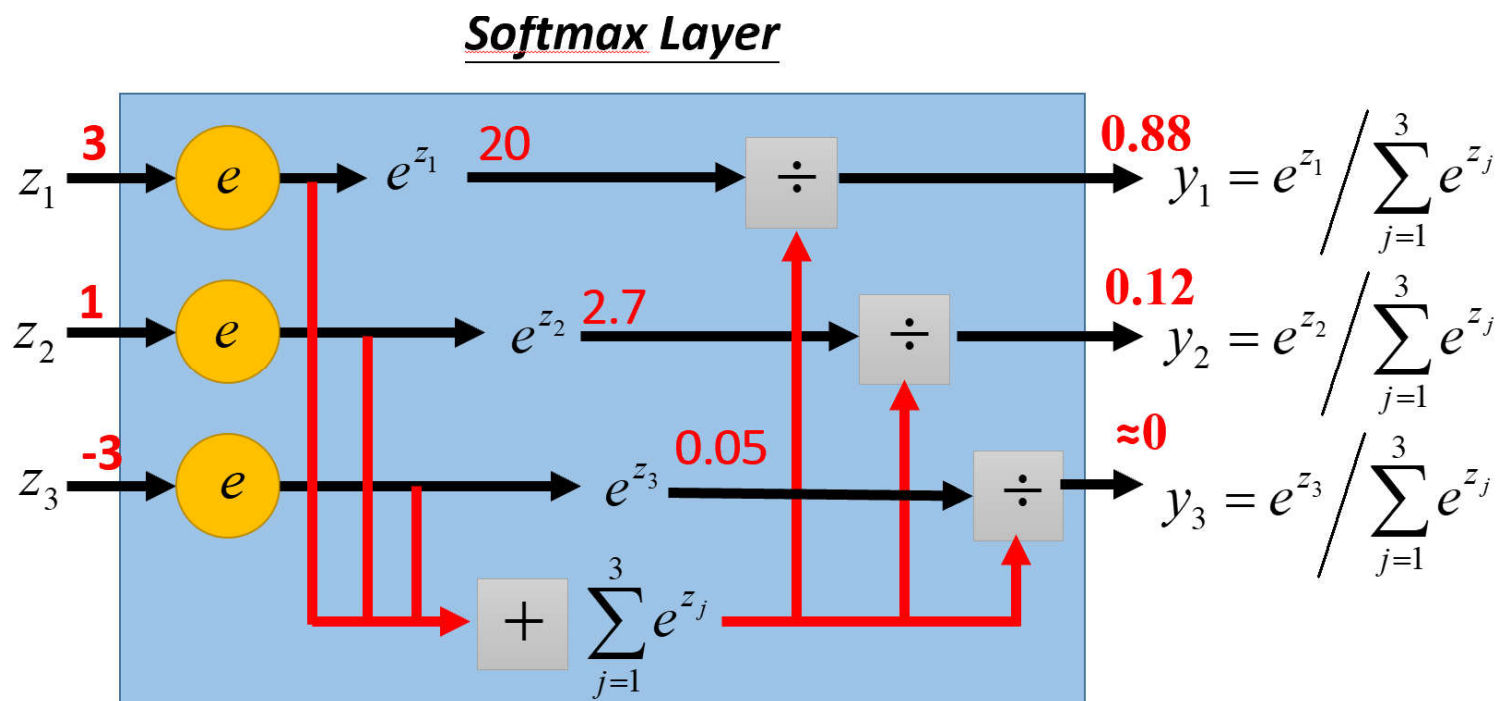
■ 分类问题

- 如果使用**Softmax**分类器，相当于网络最后一层设置**C**个神经元，其输出经过**Softmax**函数进行归一化后可以作为每个类的条件概率。

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$$

■ 最后的输出层—Softmax Layer

- 如果使用Softmax分类器，相当于网络最后一层设置C个神经元，其输出经过Softmax函数进行归一化后可以作为每个类的条件概率。
- 图像分类：将输入图像的信息转换成其属于各个类别的概率



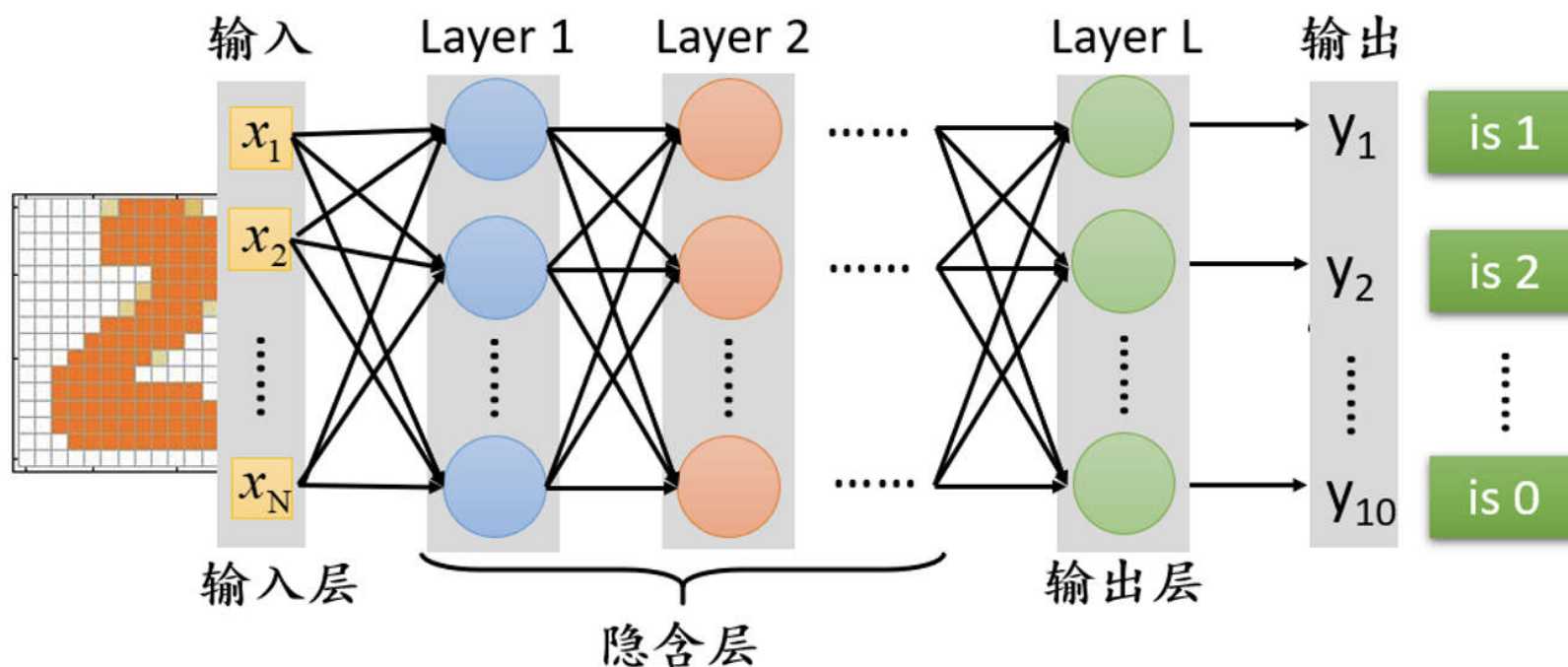
Probability:

- $0 \leq y_i \leq 1$
- $\sum_i y_i = 1$

■ 分类问题损失函数

- 采用交叉熵损失函数，对于样本(x,y)，其损失函数为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$



■ 交叉熵损失函数

- 信息量：信息是用来消除随机不确定性的东西。信息量的大小与信息发生的概率成反比。概率越大，信息量越小。概率越小，信息量越大。设某一事件发生的概率为 $P(x)$ ，其信息量表示为

$$I(x) = -\log(P(x))$$

- 信息熵：所有信息量的期望 $\mathbf{X} = (x_1, x_2, \dots, x_n)$

$$H(\mathbf{X}) = \sum_{i=1}^n P(x_i) \log(P(x_i))$$

- 相对熵（KL散度）：

$$D_{KL}(p \parallel q) = \sum_{i=1}^n p(x_i) \log\left(\frac{p(x_i)}{q(x_i)}\right)$$

↙ ↘

真实分布 预测分布

■ 交叉熵损失函数

- 交叉熵：交叉熵等于KL散度加上一个常量（信息熵），且公式相比KL散度更加容易计算，所以在机器学习中常常使用交叉熵损失函数来计算loss

$$\begin{aligned} D_{KL}(p \parallel q) &= \sum_{i=1}^n p(x_i) \log\left(\frac{p(x_i)}{q(x_i)}\right) \\ &= \sum_{i=1}^n p(x_i) \log(p(x_i)) - \sum_{i=1}^n p(x_i) \log(q(x_i)) \\ &= \boxed{-H(p(x))} + \boxed{\left[-\sum_{i=1}^n p(x_i) \log(q(x_i))\right]} \\ &\quad \text{信息熵} \qquad \qquad \text{交叉熵} \end{aligned}$$

PART THREE

梯度反向传播

Gradient Back Propagation

■ 参数学习 (Parameter Learning)

□ 给定训练集为 $D = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ ，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络，得到网络输出为 $\hat{y}^{(n)}$ ，定义在数据集 D 上的结构化风险函数为：

$$\mathcal{R}(W, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, \hat{y}^{(n)}) + \frac{1}{2} \lambda \|W\|_F^2$$

□ 梯度下降：

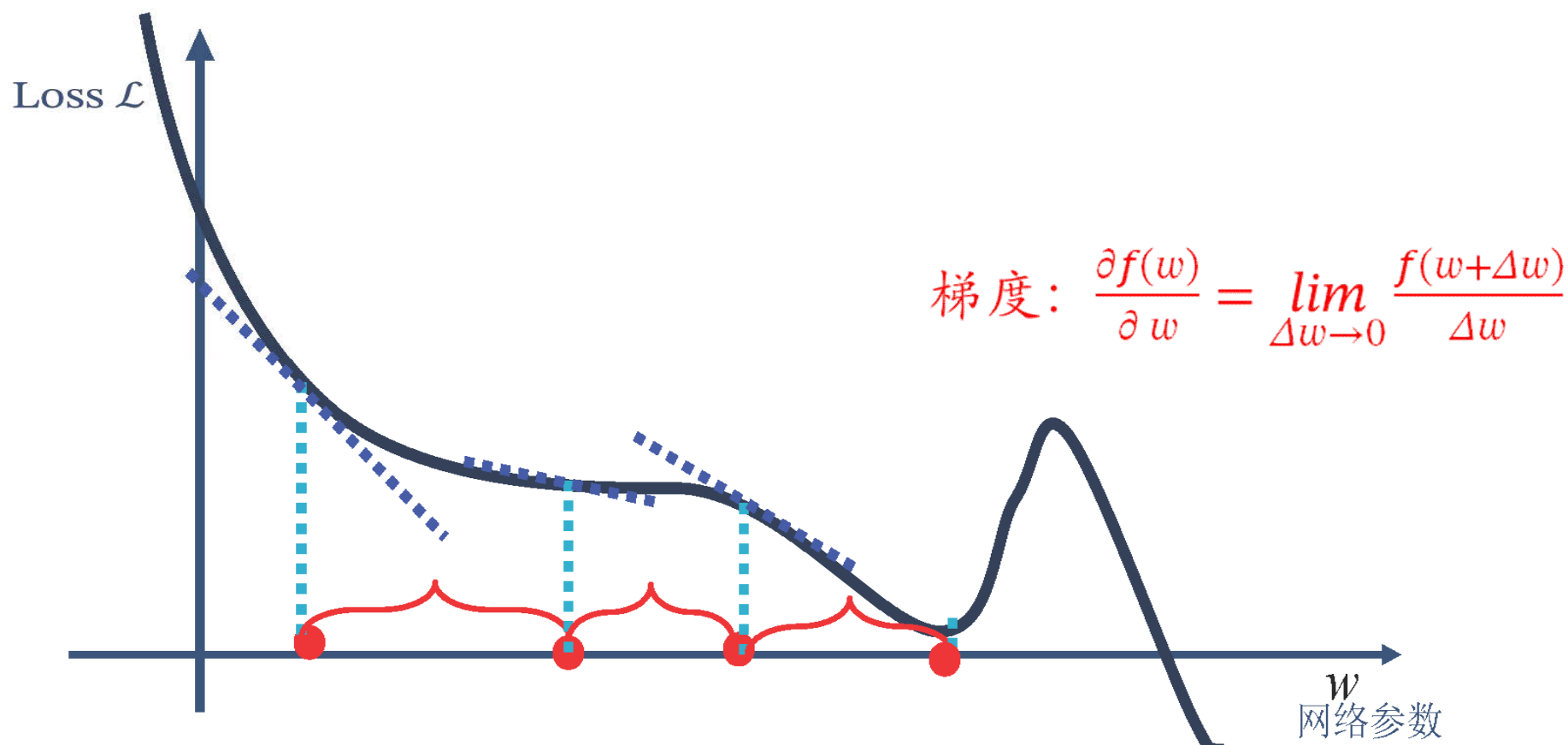
$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial W^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$



■ 梯度下降

□ 梯度下降类似盲人下山



■ 梯度下降

□ 神经网络是一个复杂的复合函数(链式法则)

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^5}{\partial f^4} \frac{\partial f^4}{\partial f^3} \frac{\partial f^3}{\partial f^2} \frac{\partial f^2}{\partial f^1} \frac{\partial f^1}{\partial x}$$

□ 梯度反向传播算法(**Back Propagation**): 根据前馈网络的特点而设计的高效方法

□ 更加通用的计算方法: 自动微分(**Automatic Differentiation**)



■ 链式法则

□ 链式法则(Chain Rule)是在微积分中求复合函数导数的一种常用方法。

(1) 若 $x \in \mathbb{R}$, $\mathbf{u} = u(x) \in \mathbb{R}^s$, $\mathbf{g} = g(\mathbf{u}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{g}}{\partial x} = \frac{\partial \mathbf{u}}{\partial x} \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \in \mathbb{R}^{1 \times t}.$$

(2) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = g(\mathbf{x}) \in \mathbb{R}^s$, $\mathbf{z} = f(\mathbf{y}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \in \mathbb{R}^{p \times t}.$$

(3) 若 $X \in \mathbb{R}^{p \times q}$ 为矩阵, $\mathbf{y} = g(X) \in \mathbb{R}^s$, $z = f(\mathbf{y}) \in \mathbb{R}$, 则

$$\frac{\partial z}{\partial X_{ij}} = \frac{\partial \mathbf{y}}{\partial X_{ij}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}.$$



■ 典型深度学习库架构



自动微分

■ 梯度下降—计算图

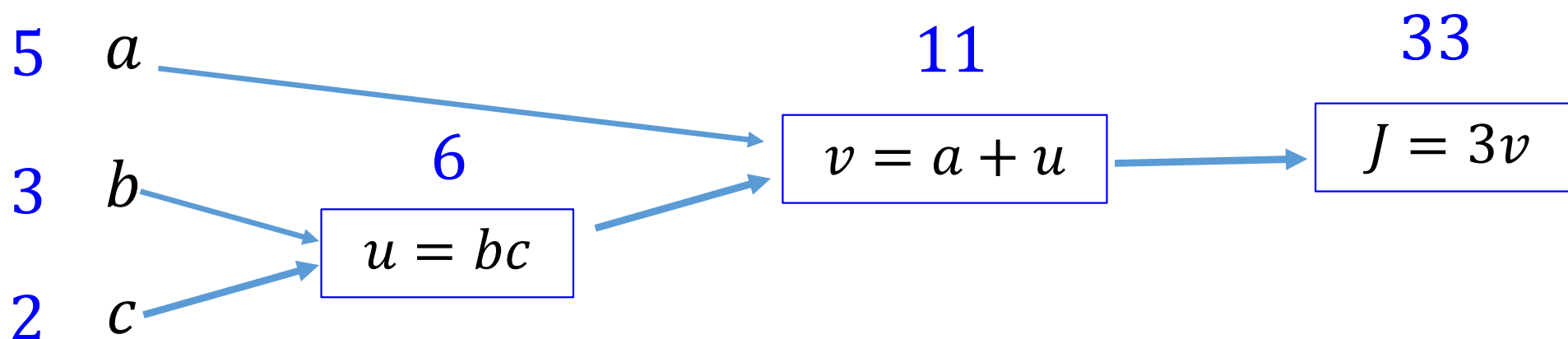
$$J(a, b, c) = 3(a + \underbrace{bc}_u) = 3(5 + 3 \times 2) = 33$$

$$u = bc$$

$$v = a + u$$

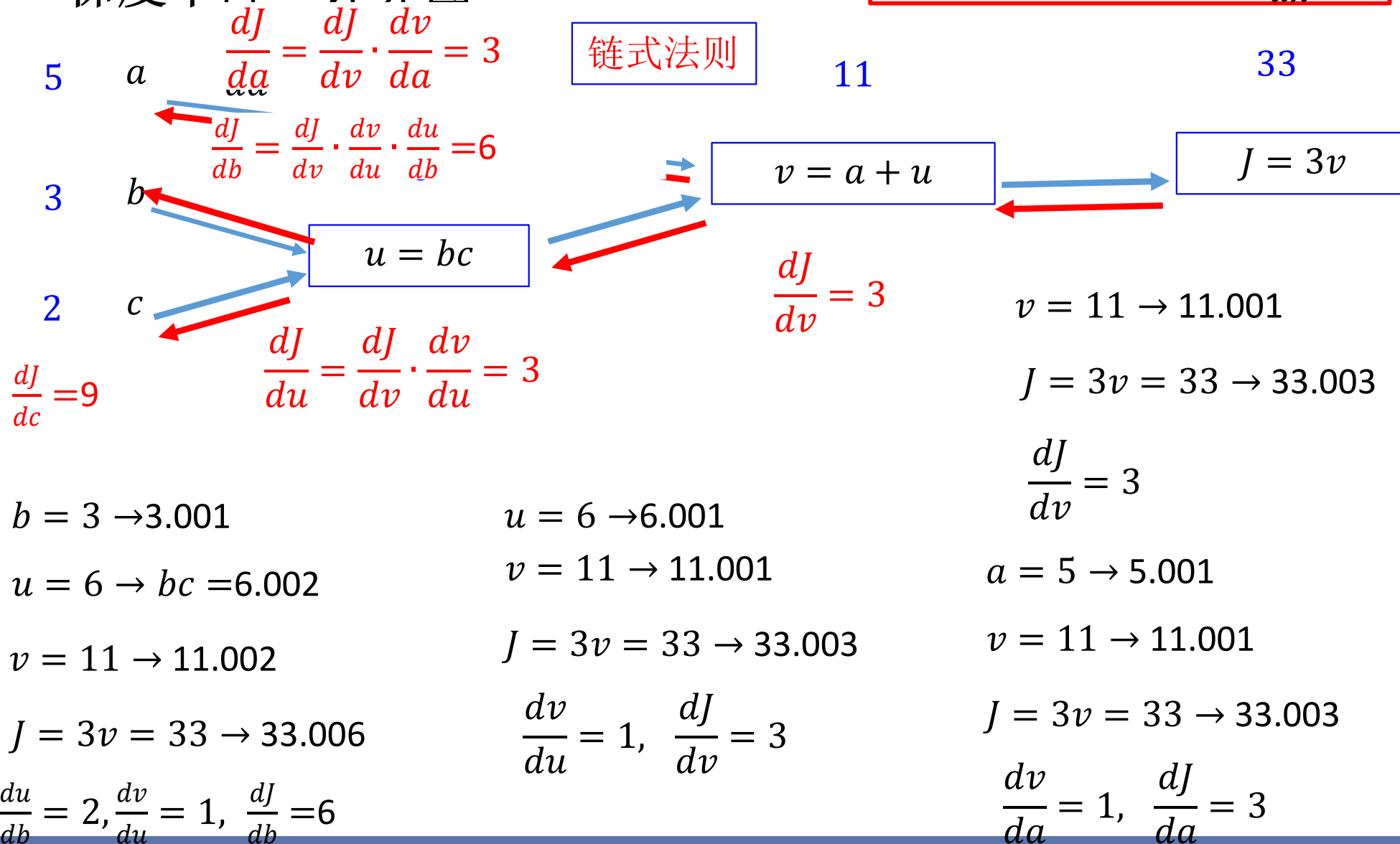
$$J = 3v$$

$$\underbrace{\underbrace{u}_v}_J$$

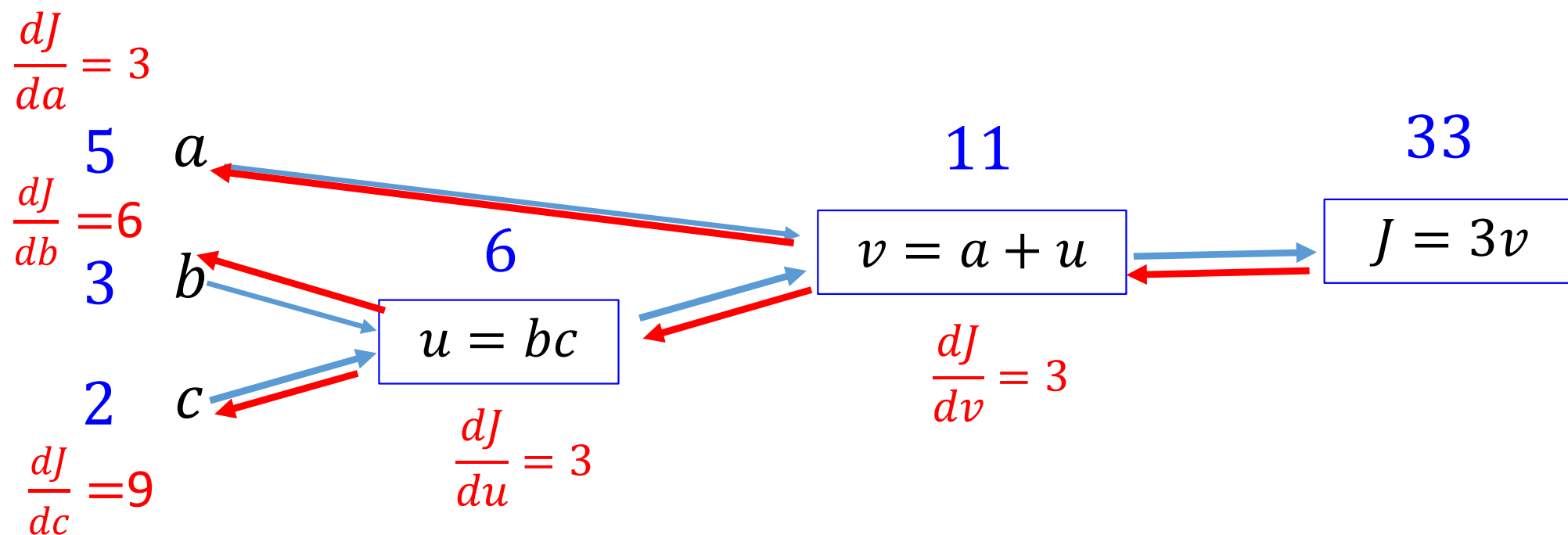


■ 梯度下降—计算图

$$y = f(x) = 3x, f'(x) = \frac{dy}{dx} = 3$$



■ 梯度下降—计算图



■ 梯度下降—计算图

□ 逻辑回归算法: $\hat{y} = f(\mathbf{w}^T \mathbf{x} + b)$, 其中 $\mathbf{z} = \mathbf{w}^T \mathbf{x} + b$, $f(\mathbf{z}) = \frac{1}{1+e^{-z}}$

□ 损失函数:

$$L(\hat{y}^{(i)}, y^{(i)}) = -y^{(i)} \log(\hat{y}^{(i)}) - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

□ 代价函数:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

□ 梯度下降法参数更新:

$$\mathbf{w} := \mathbf{w} - \eta \frac{\partial L(\mathbf{w}, b)}{\partial \mathbf{w}}, b := b - \eta \frac{\partial L(\mathbf{w}, b)}{\partial b}$$



■ 梯度下降—计算图

□ 假设单个样本, 样本只有两个特征 x_1 和 x_2 ; 参数 w_1 、 w_2 和 b :

$$z = w_1x_1 + w_2x_2 + b$$

$$\hat{y} = f(z)$$

□ 损失函数

$$L(\hat{y}, y) = -y\log(\hat{y}) - (1 - y)\log(1 - \hat{y})$$

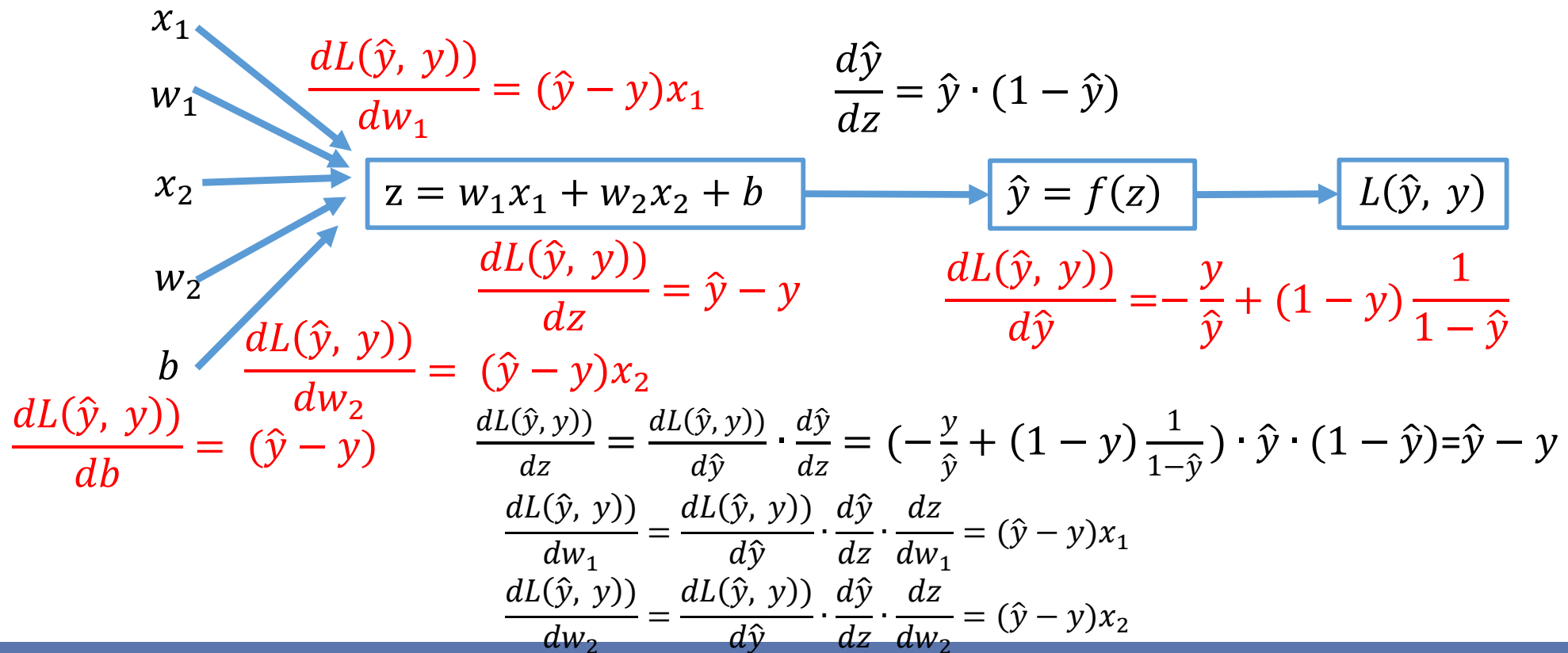
■ 梯度下降—计算图

$$z = w_1x_1 + w_2x_2 + b$$

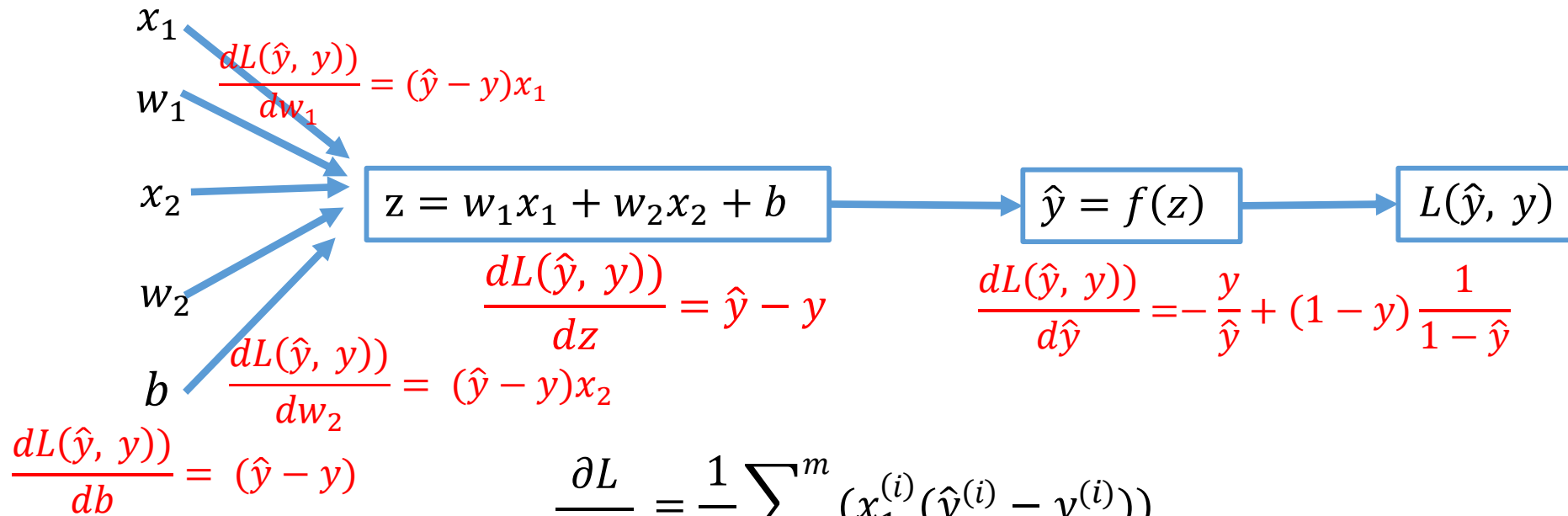
$$\hat{y} = f(z) = \frac{1}{1 + e^{-z}}$$

$$L(\hat{y}, y) = -y\log(\hat{y}) - (1 - y)\log(1 - \hat{y})$$

$$f'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = f(z)(1 - f(z))$$



■ 梯度下降—计算图



对于m个样本:

$$\frac{\partial L}{\partial w_1} = \frac{1}{m} \sum_i^m (x_1^{(i)} (\hat{y}^{(i)} - y^{(i)}))$$

$$\frac{\partial L}{\partial w_2} = \frac{1}{m} \sum_i^m (x_2^{(i)} (\hat{y}^{(i)} - y^{(i)}))$$

$$\frac{\partial L}{\partial b} = \frac{1}{m} \sum_i^m (\hat{y}^{(i)} - y^{(i)})$$

更新: $w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}, w_2 = w_2 - \eta \frac{\partial L}{\partial w_2}, b = b - \eta \frac{\partial L}{\partial b}$

■ 反向传播 (Back Propagation)

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$



■ 反向传播 (Back Propagation)

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

误差项

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \boxed{\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\boxed{\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}}$$

■ 反向传播 (Back Propagation)

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

误差项

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} &= \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \\ \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} &= \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \end{aligned}$$

权重梯度

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{m^{(l)}}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_{i:}^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

■ 反向传播 (Back Propagation)

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

误差项

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$

权重梯度

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{m^{(l)}}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_{i:}^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

偏置梯度

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

■ 反向传播 (Back Propagation)

□ 神经网络结构

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

$$\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

□ 逐层反向传播

$$\begin{aligned} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} &= \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \end{aligned}$$

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (\mathbf{W}^{(l+1)})^T$$

$$\begin{aligned} \delta^{(l)} &\triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \\ &= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (\mathbf{W}^{(l+1)})^T \cdot \delta^{(l+1)} \\ &= f'_l(\mathbf{z}^{(l)}) \odot ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}), \end{aligned}$$

■ 反向传播 (Back Propagation)

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} = \mathbb{I}_i(a_j^{(l-1)}) \delta^{(l)} = \delta_i^{(l)} a_j^{(l-1)}$$

□ 进一步， $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层权重 $\mathbf{W}^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$$

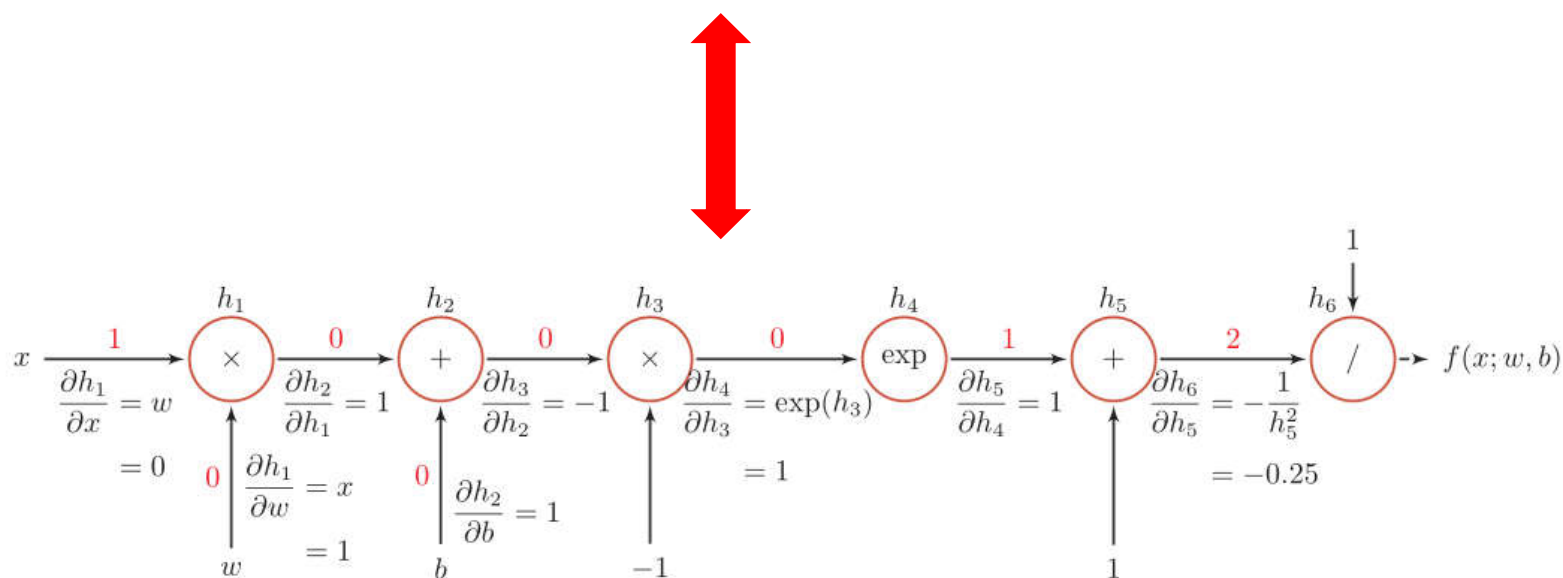
□ 同理， $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层偏置 $\mathbf{b}^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$$

■ 自动微分 (Automatic Differentiation)

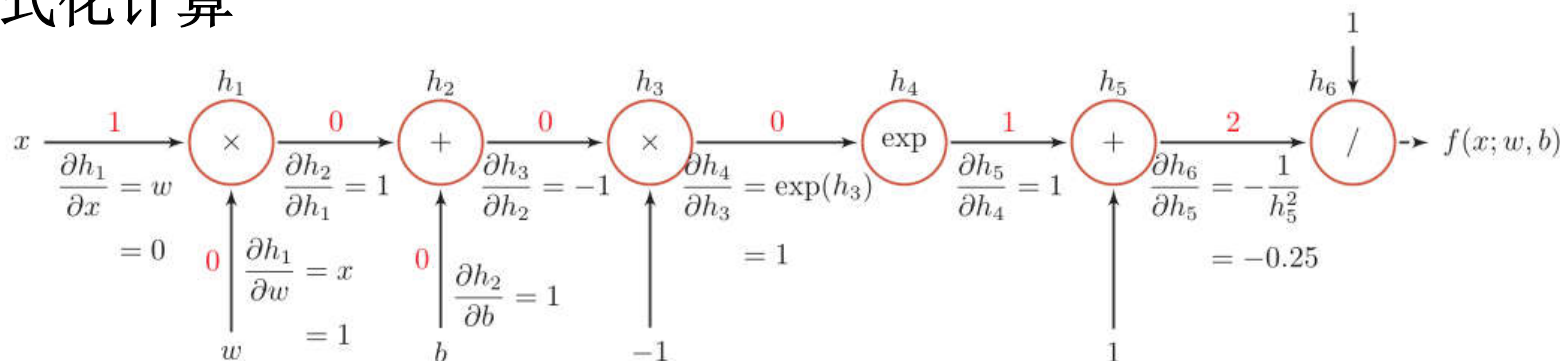
自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$



■ 自动微分 (Automatic Differentiation)

□ 形式化计算



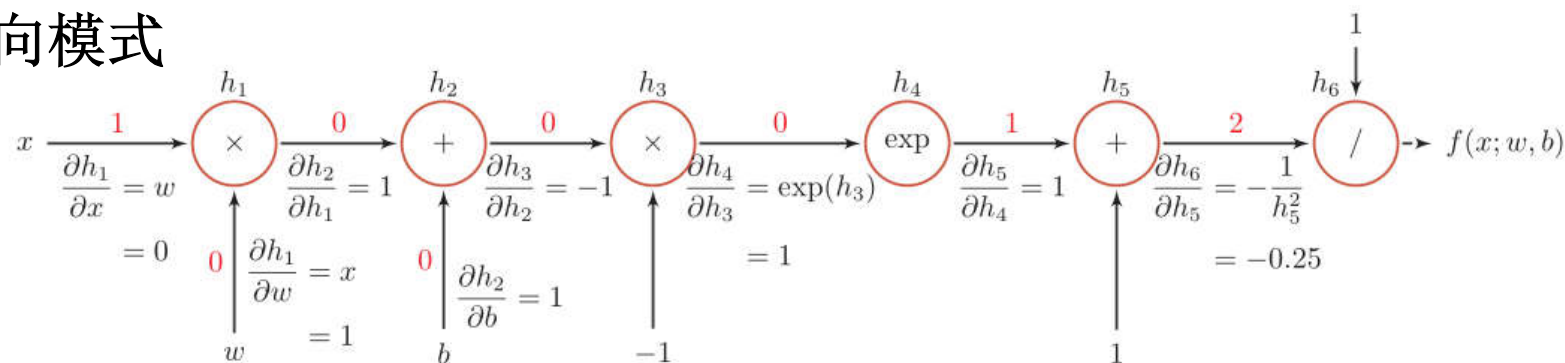
函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

当 $x = 1, w = 0, b = 0$ 时，可以得到

$$\begin{aligned}
 \frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\
 &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\
 &= 0.25.
 \end{aligned}$$

■ 自动微分 (Automatic Differentiation)

□ 前向模式



□ 反向模式 (与反向传播的计算梯度的方式相同)

前向计算每一层的状态和激活值，直到最后一层

反向计算每一层的参数的偏导数

更新参数

- 如果函数和参数之间有多条路径，可以将这多条路径上的导数再进行相加，得到最终的梯度。

■ 自动微分 (Automatic Differentiation)

□ 静态计算图

在编译时构建计算图，计算图构建好之后在程序运行时不能改变。

代表性工具箱Theano和Tensorflow

□ 动态计算图

在程序运行时动态构建，可自动根据函数的结构调整计算图。代表性工具箱DyNet, Chainer和PyTorch

□ 优缺点

静态计算图在构建时可以进行优化，并行能力强，但灵活性比较低。动态计算图则不容易优化，当不同输入的网络结构不一致时，难以并行计算，但是灵活性比较高。



■ 多层感知机

- 单层感知机（连接方式，激活函数）
- 多层感知机（等价形态，多层扩展）

■ 前馈神经网络

- 人工神经网络
- 前馈神经网络（连接模式，数据传递）
- 分类问题

■ 梯度反向传播

- 参数学习
- 梯度下降（链式法则，反向传播，自动微分）

基于全连接网络的手写数字识别

```
class FCNet(nn.Module):  
    def __init__(self, input_size, hidden_size, output_size):  
        super(FCNet, self).__init__()  
        self.fc1 = nn.Linear(input_size, hidden_size)  
        self.relu = nn.ReLU()  
        self.fc2 = nn.Linear(hidden_size, output_size)
```

```
    def forward(self, x):  
        out = self.fc1(x)  
        out = self.relu(out)  
        out = self.fc2(out)  
        return out
```

```
class student(object):  
    def __init__(self, name, age, score):  
        self.name = name  
        self.age = age  
        self.score = score  
  
    def print_age(self):  
        print("%s's name: %s" % (self.name, self.age))  
  
    def print_score(self):  
        print("%s's age: %s" % (self.name, self.score))
```

```
stu1 = student('Amy', 13, 86)  
stu2 = student('Tom', 15, 79)
```



```
class FCNet(nn.Module):  
    def __init__(self, input_size, hidden_size, output_size):  
        super(FCNet, self).__init__()  
        self.fc1 = nn.Linear(input_size, hidden_size)  
        self.relu = nn.ReLU()  
        self.fc2 = nn.Linear(hidden_size, output_size)  
  
    def forward(self, x):  
        out = self.fc1(x)  
        out = self.relu(out)  
        out = self.fc2(out)  
        return out
```

基于全连接网络的手写数字识别

```
class MyTestData(Dataset):  
    def __init__(self, params):  
        super(MyTestData, self).__init__()  
        self.test_im_path = params['test_im_path']  
        self.test_lb_path = params['test_lb_path']  
        self.test_im_num = 10000  
        self.test_labels = open(self.test_lb_path, 'r').read().splitlines()
```

初始化
参数设置

```
    def __len__(self):  
        return self.test_im_num
```

```
    def __getitem__(self, index):  
        # load image  
        img_file = os.path.join(self.test_im_path, str(index) + '.png')  
        img = Image.open(img_file)  
        im = self.transform(img)  
  
        # load label  
        lb = int(self.test_labels[index])  
  
        return im, lb
```

读入图像和标签

```
    def transform(self, img):  
        transform_img = transforms.Compose([transforms.Resize((28, 28)),  
                                             transforms.ToTensor()])  
        im = transform_img(img)  
        return im
```

对图像进行变换
(转化为tensor)



```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
# 参数设置
```

```
params = {}  
params['train_im_path'] = 'D:\\myproject\\mnist_pytorch\\mnist\\train-images\\'  
params['test_im_path'] = 'D:\\myproject\\mnist_pytorch\\mnist\\t10k-images\\'  
params['train_lb_path'] = 'D:\\myproject\\mnist_pytorch\\mnist\\train-labels.txt'  
params['test_lb_path'] = 'D:\\myproject\\mnist_pytorch\\mnist\\t10k-labels.txt'  
params['INPUT_SIZE'] = 784  
params['HIDDEN_SIZE'] = 500  
params['OUTPUT_SIZE'] = 10  
params['NUM_EPOCHES'] = 10  
params['BATCH_SIZE'] = 100  
params['LEARNING_RATE'] = 0.001
```

参数设置

```
# load dataset
```

```
train_loader = DataLoader(MyData(params),  
                           shuffle=True,  
                           batch_size=params['BATCH_SIZE'])  
test_loader = DataLoader(MyTestData(params),  
                          shuffle=False,  
                          batch_size=1)
```

加载训练数据和测试数据

定义网络

```
model = FCNet(params['INPUT_SIZE'], params['HIDDEN_SIZE'], params['OUTPUT_SIZE']).to(device)
```

```
criterion = nn.CrossEntropyLoss()
```

定义损失函数(交叉熵损失函数)

```
optimizer = torch.optim.Adam(model.parameters(), lr=params['LEARNING_RATE'])
```

定义优化方法(ADAM)



■ 网络训练

```
for epoch in range(params['NUM_EPOCHES']):
    for i, (images, labels) in enumerate(train_loader):
        # Move tensors to the configured device
        images = images.reshape(-1, 28*28).cuda(device=0)  ## -1 是指模糊控制的意思，即固定784列，不知道多少行
        labels = labels.cuda(device=0)
        # images = images.reshape(-1, 28 * 28).to(device)  ## -1 是指模糊控制的意思，即固定784列，不知道多少行
        # labels = labels.to(device)

        # 前向传播
        outputs = model(images)
        loss = criterion(outputs, labels)

        # 反向传播和优化
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    if (i+1) % 100 == 0:
        print('Epoch [{}/{}], Step [{}/{}], Loss: {:.4f}'
              .format(epoch+1, params['NUM_EPOCHES'], i+1, total_step, loss.item()))
        losses.append(loss.item())
```

```
# 测试模型
# 在测试阶段，不用计算梯度
with torch.no_grad():
    correct = 0
    total = 0
    model = model.eval()
    for images, labels in test_loader:
        images = images.reshape(-1, 28 * 28).to(device)
        labels = labels.to(device)
        outputs = model(images)
        _, predict = torch.max(outputs.data, 1)
        ##这里返回两组数据，最大image_data和最大值索引，可以用torch.argmax()更为直观；这里去理解其
        ## 这个 _ , predicted是python的一种常用的写法，表示后面的函数其实会返回两个值
        # 但是我们对第一个值不感兴趣，就写个_在那里，把它赋值给_就好，我们只关心第二个值predicted
        # 比如 _ ,a = 1,2 这中赋值语句在python中是可以通过的，你只关心后面的等式中的第二个位置的值是
        total += labels.size(0) ##更新测试图片的数量 size(0),返回行数
        correct += (predict == labels).sum().item()

accuracy = correct / total
accs.append(accuracy)
print('Accuracy of the network on the 10000 test images: {} %'.format(100 * accuracy))
```

■ 保存网络模型

```
# 保存模型  
torch.save(model.state_dict(), 'model.pth')
```

```
Epoch [1/10], Step [100/600], Loss: 0.6180
Epoch [1/10], Step [200/600], Loss: 0.2788
Epoch [1/10], Step [300/600], Loss: 0.3504
Epoch [1/10], Step [400/600], Loss: 0.2611
Epoch [1/10], Step [500/600], Loss: 0.3334
Epoch [1/10], Step [600/600], Loss: 0.1110
Accuracy of the network on the 10000 test images: 93.2 %
Epoch [2/10], Step [100/600], Loss: 0.2007
Epoch [2/10], Step [200/600], Loss: 0.1345
Epoch [2/10], Step [300/600], Loss: 0.1862
Epoch [2/10], Step [400/600], Loss: 0.2383
Epoch [2/10], Step [500/600], Loss: 0.1317
Epoch [2/10], Step [600/600], Loss: 0.1385
Accuracy of the network on the 10000 test images: 95.5 %
Epoch [3/10], Step [100/600], Loss: 0.0772
Epoch [3/10], Step [200/600], Loss: 0.1712
Epoch [3/10], Step [300/600], Loss: 0.1935
Epoch [3/10], Step [400/600], Loss: 0.2046
Epoch [3/10], Step [500/600], Loss: 0.0839
Epoch [3/10], Step [600/600], Loss: 0.2274
Accuracy of the network on the 10000 test images: 95.67999999999999 %
Epoch [4/10], Step [100/600], Loss: 0.1628
Epoch [4/10], Step [200/600], Loss: 0.0509
Epoch [4/10], Step [300/600], Loss: 0.1581
Epoch [4/10], Step [400/600], Loss: 0.0813
Epoch [4/10], Step [500/600], Loss: 0.1619
Epoch [4/10], Step [600/600], Loss: 0.0967
Accuracy of the network on the 10000 test images: 96.12 %
Epoch [5/10], Step [100/600], Loss: 0.0991
Epoch [5/10], Step [200/600], Loss: 0.0900
Epoch [5/10], Step [300/600], Loss: 0.1194
Epoch [5/10], Step [400/600], Loss: 0.1203
Epoch [5/10], Step [500/600], Loss: 0.0709
Epoch [5/10], Step [600/600], Loss: 0.1261
Accuracy of the network on the 10000 test images: 96.72 %
Epoch [6/10], Step [100/600], Loss: 0.0240
Epoch [6/10], Step [200/600], Loss: 0.0739
Epoch [6/10], Step [300/600], Loss: 0.0865
Epoch [6/10], Step [400/600], Loss: 0.0295
Epoch [6/10], Step [500/600], Loss: 0.0645
Epoch [6/10], Step [600/600], Loss: 0.1076
```



```
# plot loss figure and accuracy figure
plt.figure()
plt.title('loss')
plt.plot(losses)

plt.figure()
plt.title('accuracy')
plt.plot(accs)

plt.show()

# 保存模型
torch.save(model.state_dict(), 'model.pth')
```

基于全连接网络的手写数字识别

